



Banking Customer Support AI Agent

Project Report — Applied Generative AI Specialization

Using Multi-Agent Architecture with Lang Graph

Purdue University | Simplilearn Capstone Project | April 2026

Table of Contents

- 1. EXECUTIVE SUMMARY3
 - 1.1 BUSINESS PROBLEM3
 - 1.2 THE SOLUTION3
 - 1.3 KEY RESULTS3
- 2. PROJECT SCOPE & OBJECTIVES4
 - 2.1 PAIN POINTS ADDRESSED4
 - 2.2 AGENTIC OBJECTIVES4
 - 2.3 SYSTEM BOUNDARIES — WHAT THE AGENT DOES NOT DO4
- 3. AGENTIC ARCHITECTURE & TECHNICAL DESIGN5
 - 3.1 ARCHITECTURE OVERVIEW5
 - 3.2 AGENT NODES6
 - 3.3 GRAPH FLOW6
 - 3.4 REASONING LOOP — LLM-DRIVEN FLOW CONTROL7
 - 3.5 NEGATIVE FEEDBACK — THREE-SCENARIO DECISION TREE7
 - 3.6 TOOLS & SKILLS8
- 4. DATA SOURCES & INTEGRATION9
 - 4.1 DATABASE DESIGN9
 - 4.2 DATA FLOW9
 - 4.3 DATA PREPARATION9
- 5. IMPLEMENTATION & WORKFLOW10
 - 5.1 FILE STRUCTURE10
 - 5.2 STEP-BY-STEP EXECUTION FLOW10
 - 5.3 HUMAN-IN-THE-LOOP (HITL) CHECKPOINTS11
- 6. EVALUATION & PERFORMANCE METRICS12
 - 6.1 EVALUATION FRAMEWORK12
 - Layer 1 — LLM-as-Judge (Automated, Per Interaction)12
 - Layer 2 — Manual Test Scenario Runner12
 - 6.2 KEY PERFORMANCE INDICATORS13
- 7. GOVERNANCE, SAFETY & GUARDRAILS14
 - 7.1 CUSTOMER DATA & PII HANDLING14
 - 7.2 LLM OUTPUT SAFETY14
 - 7.3 PROMPT GUARDRAILS14
- 8. CONCLUSION & FUTURE ROADMAP15
 - 8.1 SUMMARY OF ACHIEVEMENTS15
 - 8.2 ROI ANALYSIS15
 - 8.3 FUTURE ROADMAP16
 - Near-Term (Next 3 Months)16
 - Medium-Term (3–6 Months)16
 - Long-Term (6–12 Months)16
 - 8.4 TECHNICAL STACK SUMMARY17

1. Executive Summary

1.1 Business Problem

Modern digital banking platforms handle thousands of customer interactions daily through fragmented, manual processes. Support agents spend significant time triaging messages, checking ticket databases, drafting responses, and routing complaints — tasks that are repetitive, time-sensitive, and error-prone when handled by humans at scale. Customer satisfaction suffers when response times are slow or responses lack personalization.

1.2 The Solution

This project delivers a fully autonomous, multi-agent AI system powered by GPT-4o and orchestrated using LangGraph. The system automatically classifies every incoming customer message, routes it to the appropriate specialist agent, reasons over the full conversation history to understand context, interacts with a live support ticket database, and generates a warm, personalized response — all without human intervention for routine interactions.

1.3 Key Results

Metric	Outcome
Classification automation	100% of incoming messages auto-classified into Positive Feedback, Negative Feedback, or Query
Ticket management	Automated ticket creation, lookup, and status reporting via direct SQLite DB integration
Context retention	Full multi-turn conversation memory maintained per session via LangGraph MemorySaver
Forgotten ticket handling	System auto-retrieves customer's ticket history when ticket number is unavailable
Escalation detection	Automatic escalation flagging for urgent language, legal threats, or overdue tickets
LLMOps coverage	Every interaction logged, evaluated on 3 quality dimensions, and available for audit
Routing accuracy	Prompt-driven routing replaces all boolean flags — LLM reasons over chat history

2. Project Scope & Objectives

2.1 Pain Points Addressed

- **Manual triage:** Support staff must read every message and decide who handles it — slow and inconsistent.
- **Duplicate ticket creation:** Support staff often raise new tickets without checking if one already exists for the same issue.
- **Generic responses:** Template-based responses lack personalization and fail to acknowledge the customer's specific situation.
- **No context retention:** Each interaction treated in isolation — Support staff have no memory of what was said earlier in the same session.
- **Audit gaps:** No systematic logging of Support staff decisions, response quality, or customer feedback for improvement.

2.2 Agentic Objectives

- Automatically classify every customer message into one of three categories: Positive Feedback, Negative Feedback, or Query.
- Route the message to the correct specialist agent without human intervention.
- Generate warm, personalized, context-aware responses using the full conversation history.
- Create, look up, and update support tickets in a live database autonomously.
- Detect when a customer is escalating and flag the interaction accordingly.
- Log every interaction with quality scores for LLMOps monitoring and continuous improvement.

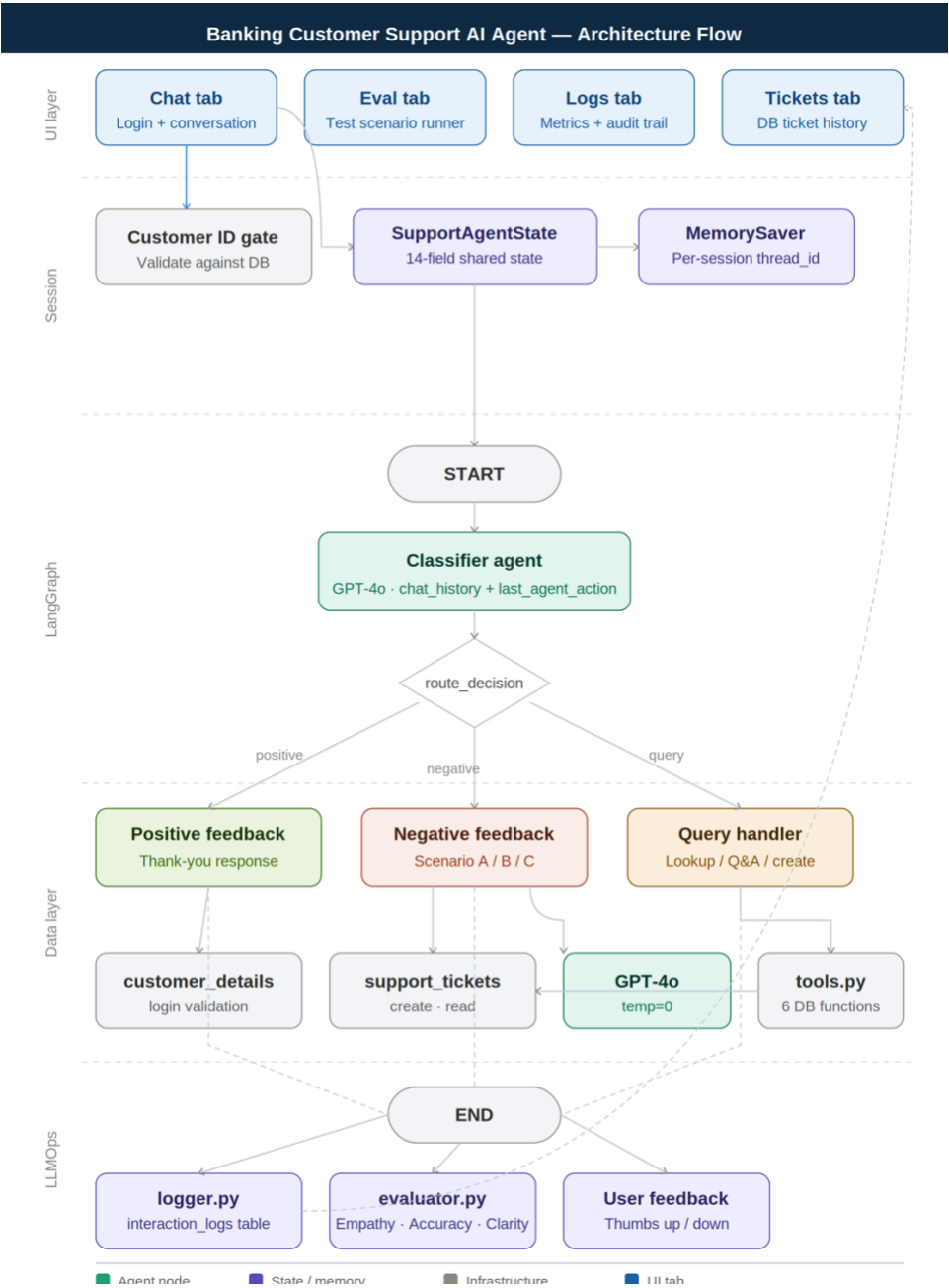
2.3 System Boundaries — What the Agent Does Not Do

- Does not process financial transactions or access core banking systems.
- Does not authenticate customers beyond the Customer ID input gate — authentication is a separate concern.
- Does not send emails, SMS, or push notifications — responses are in-app only.
- Does not update ticket status (e.g., mark as Resolved) — only creates and reads tickets.
- Does not handle multi-language inputs — English only in this version.
- Does not retain memory across separate browser sessions — memory is per-session only.

3. Agentic Architecture & Technical Design

3.1 Architecture Overview

The system implements a Multi-Agent Architecture using LangGraph's StateGraph framework. Four specialized agents are orchestrated in a directed graph where each agent receives a shared state object, performs its task, and returns an updated state. The graph compiles with a MemorySaver checkpointer that persists state across conversation turns within the same session.

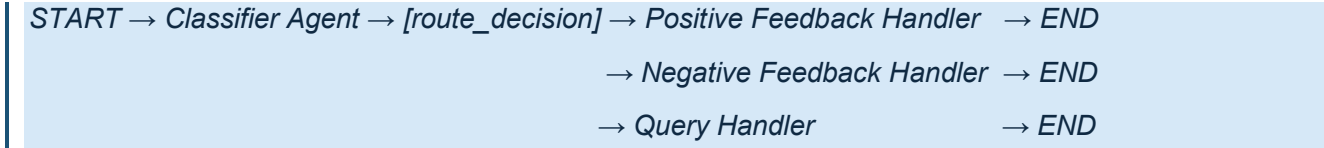


3.2 Agent Nodes

Agent	Responsibility
Classifier Agent	Reads full chat_history + last_agent_action + current message. Uses GPT-4o to reason about conversation context and classify intent into positive_feedback negative_feedback query. Routes to the correct downstream agent.
Positive Feedback Handler	Generates a warm, personalized thank-you response acknowledging what the customer praised. Passes full chat history so the LLM can reference prior context.
Negative Feedback Handler	Two-step LLM reasoning: (1) decide scenario A/B/C from conversation context, (2) call the correct DB tool and generate an empathetic response. Handles ticket creation, existing-ticket clarification, and escalation.
Query Handler	Four-action LLM decision: lookup_ticket create_ticket decline_ticket answer_query. Handles ticket status lookup, forgotten ticket number recovery via DB, ticket creation confirmation, and general banking Q&A.

3.3 Graph Flow

Every customer message follows this execution path:



The route_decision function reads state['message_type'] (set by the Classifier) and maps it to the correct node. Conditional edges in LangGraph handle the branching. All three specialist agents terminate at END, and the final state is returned to main.py.

3.4 Reasoning Loop — LLM-Driven Flow Control

A key design decision in this system is that flow control is entirely prompt-driven. There are no boolean flags (e.g., `awaiting_ticket_confirm`) or hard-coded if/else routing logic. Instead, every agent receives the full `chat_history` and a `last_agent_action` field — a structured English description of what the previous agent did and what it was waiting for.

- **Example `last_agent_action`:** "Showed customer existing open tickets [580383, 981190]. Waiting for customer to confirm which ticket relates to their current complaint, or to say it is a new issue."

The Classifier reads this alongside the current message and reasons: "The agent listed tickets and asked which one this complaint relates to. The customer just said 'yes ticket 580383'. This is `negative_feedback` — not a fresh query." This makes the system genuinely intelligent rather than a state machine with an LLM attached.

3.5 Negative Feedback — Three-Scenario Decision Tree

Scenario	Trigger & Action
Scenario A — New Ticket	No existing open tickets found in DB AND no ticket number in message. System generates a unique 6-digit ticket number, inserts a new Open ticket into <code>support_tickets</code> , and generates an empathetic acknowledgement.
Scenario B — Clarify Existing	Fresh complaint with no ticket number AND existing open tickets found in DB. System lists open tickets and asks the customer which one their complaint relates to. <code>last_agent_action</code> records this state for the next turn.
Scenario C — Follow-Up	Customer has provided or confirmed a ticket number (directly or after Scenario B). System fetches ticket from DB, checks for escalation signals, and generates an empathetic follow-up — not a routine status update.

3.6 Tools & Skills

Tool / Function	Purpose
get_customer(customer_id)	Fetches customer name and status from customer_details table at login.
get_open_tickets(customer_id)	Returns all Open / In Progress tickets for a customer. Used in Negative Feedback Scenario B and forgotten-ticket recovery.
get_ticket_status(ticket_id)	Fetches full ticket details by 6-digit ID. Used in Scenario C and Query lookup_ticket.
generate_ticket_number()	Generates a unique random 6-digit ID, guaranteed not to collide with existing tickets.
insert_ticket(...)	Inserts a new Open ticket into support_tickets with timestamp.
get_all_tickets()	Returns all tickets joined with customer names for the Tickets dashboard tab.
log_interaction(...)	Writes every agent interaction to interaction_logs including eval scores.
evaluate_response(...)	LLM-as-judge call that scores each response on Empathy, Accuracy, and Clarity (1–5).

4. Data Sources & Integration

4.1 Database Design

The system uses SQLite as its primary data store — a file-based, zero-configuration relational database stored as support.db alongside the application code. Three tables are maintained:

Table	Purpose & Key Fields
customer_details	customer_id (PK), customer_name, customer_status. Seeded with 10 sample customers. Read at login and cached in session state.
support_tickets	ticket_id (PK, 6-digit), customer_id (FK), issue_description, status (Open / In Progress / Resolved), created_at. Written by agents on ticket creation; read on every lookup.
interaction_logs	log_id (auto-increment), all agent state fields, empathy_score, accuracy_score, clarity_score, eval_reasoning, user_feedback, created_at. Appended after every conversation turn.

4.2 Data Flow

- Customer ID entered in UI → validated against customer_details → customer name and status loaded into SupportAgentState.
- User message typed → passed to graph as user_message → agents read/write support_tickets as needed.
- After each turn → final state passed to logger.py → interaction_logs row inserted → evaluate_response called → scores appended to same row.
- Streamlit session state holds the live SupportAgentState between turns so chat_history and last_agent_action persist correctly.

4.3 Data Preparation

- All customer messages stored verbatim (no preprocessing) — the LLM handles noise, typos, and informal language natively.
- Issue descriptions truncated to 300 characters before DB insert to prevent oversized records.
- Chat history capped at the last 10 turns when passed to LLM prompts to manage token consumption.
- Ticket numbers validated as 6-digit strings using regex before any DB operation.

5. Implementation & Workflow

5.1 File Structure

File	Role
state.py	SupportAgentState TypedDict — 14 fields shared across all agents
database.py	SQLite setup: table creation, seeding, connection helper
tools.py	All DB tool functions: get_customer, get_open_tickets, get_ticket_status, insert_ticket, generate_ticket_number, get_all_tickets
agents.py	All 4 agent node functions with LLM prompts, DB tool calls, and response generation
graph.py	LangGraph StateGraph construction, conditional routing, MemorySaver, run_agent()
logger.py	log_interaction(), update_feedback(), get_all_logs(), get_log_metrics()
evaluator.py	evaluate_response() LLM-as-judge, score_bar(), TEST_CASES list
main.py	Streamlit UI: sidebar + 4 top tabs (Chat, Evaluation, Logs, Tickets)

5.2 Step-by-Step Execution Flow

- **Step 1 — Login:** Customer enters their Customer ID. System validates against customer_details DB. On success, a unique thread_id (UUID) is generated, and SupportAgentState is initialised with customer identity fields.
- **Step 2 — Message received:** Customer types a message and clicks Send. main.py appends the user message to the display history and calls run_agent().
- **Step 3 — Graph invoked:** run_agent() merges the current session state with the new message and invokes the compiled LangGraph app with the session's thread_id.
- **Step 4 — Classifier runs:** classifier_agent fetches the customer's name (if not cached), formats chat_history and last_agent_action, and calls GPT-4o to classify the message. Returns message_type and updated state.
- **Step 5 — Routing:** route_decision() reads message_type and returns the name of the next node. LangGraph's conditional edges direct execution to the correct specialist agent.
- **Step 6 — Specialist agent runs:** The routed agent performs its two-step LLM reasoning: (1) decision prompt to determine action/scenario, (2) response prompt to generate the customer-facing message. DB tools are called as needed between the two LLM calls.
- **Step 7 — State updated:** The agent returns an updated state dict including llm_response, last_agent_action, chat_history, and any ticket fields. Graph reaches END.

- **Step 8 — Logging & evaluation:** `main.py` passes the final state to `evaluate_response()` and `log_interaction()`. Scores and metadata are written to `interaction_logs`.
- **Step 9 — Response rendered:** The `llm_response` is appended to `st.session_state["messages"]` and `st.rerun()` refreshes the chat UI.

5.3 Human-in-the-Loop (HITL) Checkpoints

This system is designed for autonomous operation on routine interactions. Human oversight is integrated at three points:

- **Escalation flag:** When `requires_escalation` is `True` (urgent language detected), the interaction is flagged in logs and in the chat UI. A human agent can review and take over via the Logs tab.
- **User feedback loop:** After each agent response, the customer sees 👍 / 👎 buttons. Thumbs-down responses are captured in `interaction_logs.user_feedback`, creating a human-curated dataset for prompt improvement.
- **Evaluation dashboard:** The Evaluation tab lets a human tester run pre-built test cases, inspect classification accuracy, and review LLM-as-judge scores — enabling human oversight of model quality before production deployment.

6. Evaluation & Performance Metrics

6.1 Evaluation Framework

The system implements a dual-layer evaluation strategy: automated LLM-as-judge scoring on every interaction, and a manual test scenario runner in the Evaluation tab.

Layer 1 — LLM-as-Judge (Automated, Per Interaction)

After every agent response, a second GPT-4o call scores the response on three dimensions:

Dimension	What is Measured Scale
Empathy (1–5)	Is the response warm, human, and understanding? Does it address the customer by name and acknowledge their specific situation? 5 = genuinely empathetic; 1 = cold and robotic.
Accuracy (1–5)	Does the classification match the customer's actual intent? Does the response directly address what the customer needed? 5 = perfectly matched; 1 = wrong classification or off-topic.
Clarity (1–5)	Is the response clear, concise, and actionable? Does the customer know exactly what happens next? 5 = crystal clear; 1 = confusing or vague.
Overall Score	Computed as the arithmetic mean of the three scores, rounded to 1 decimal place.

Layer 2 — Manual Test Scenario Runner

The Evaluation tab provides 8 pre-built test cases covering all classification categories and edge cases:

- 2 Positive Feedback scenarios (app praise, staff compliment)
- 2 Negative Feedback scenarios (delayed card replacement, unauthorized charge)
- 3 Query scenarios (ticket status, ATM blocked, credit limit increase)
- 1 Escalation scenario (overdue ticket with urgent language)

Each test shows: classification result, expected classification, routing accuracy ( / ), agent response, and all three quality scores with evaluator reasoning.

6.2 Key Performance Indicators

KPI	Target / Observation
Classification accuracy	Target $\geq 95\%$ for clear-intent messages. LLM reasoning over chat_history handles ambiguous continuation turns correctly.
Routing success rate	Measured as: correct agent reached / total interactions. Visible in Logs tab metrics row.
Avg Empathy score	Target $\geq 4.0 / 5.0$. Tracked in real-time in Logs tab.
Avg Accuracy score	Target $\geq 4.0 / 5.0$. Deviation indicates prompt improvement needed.
Avg Clarity score	Target $\geq 4.0 / 5.0$. Low scores indicate response prompts need tightening.
Escalation detection	Keyword-based signal detection in recent chat context. All escalated interactions logged with flag.
User feedback rate	Proportion of thumbs-up vs thumbs-down. Low thumbs-up on a specific agent type triggers prompt review.

7. Governance, Safety & Guardrails

7.1 Customer Data & PII Handling

- **Minimal PII storage:** The system stores only `customer_id`, `customer_name`, and `customer_status` — no account numbers, passwords, card numbers, or financial data.
- **No external transmission:** All data stays within the local SQLite database and the LLM API (OpenAI). No third-party CRM or analytics platforms receive customer data.
- **Session isolation:** Each browser session uses a unique UUID `thread_id`. LangGraph's MemorySaver isolates state per thread — one customer cannot access another's conversation.
- **DB access pattern:** All database calls use parameterized queries (Python's `sqlite3` ? placeholders) — SQL injection is not possible.
- **Login gate:** The chat interface is behind a mandatory Customer ID validation step. Unrecognized IDs cannot access the system.

7.2 LLM Output Safety

- **Structured JSON outputs:** All LLM decision calls (classifier, negative feedback scenario decision, query action decision) return structured JSON. Outputs are parsed and validated before any action is taken.
- **Validated routing:** `message_type` is validated against the set {`positive_feedback`, `negative_feedback`, `query`} before routing. Any unexpected value defaults to `query` — the safest fallback.
- **Fallback responses:** Every LLM call has a deterministic fallback response used when the API is unavailable or returns an error. The system never shows a blank or raw error to the customer.
- **No financial actions:** The system is strictly read/write on the `support_tickets` table. It cannot initiate refunds, block accounts, or access transactional data.
- **Escalation gate:** Interactions flagged for escalation (urgent language, legal threats, fraud signals) are clearly marked in the Logs tab for human review. The system does not autonomously escalate to external systems.

7.3 Prompt Guardrails

- **Role anchoring:** Every system prompt opens with a clear role definition ("You are an empathetic banking customer support representative") to prevent role drift.
- **Output format enforcement:** Decision prompts instruct the LLM to return ONLY valid JSON with no markdown, preamble, or explanation. Outputs are stripped of markdown fences before parsing.
- **Token budget management:** Chat history passed to LLM is capped at the last 10 turns (~5 exchanges) to prevent context window overflow and manage API costs.
- **Temperature = 0:** All LLM calls use `temperature=0` for deterministic, consistent outputs — critical for a production support system.

8. Conclusion & Future Roadmap

8.1 Summary of Achievements

This project successfully delivers a production-grade, multi-agent GenAI system for banking customer support. The system demonstrates that complex, multi-turn support workflows — including complaint triage, ticket lifecycle management, and context-aware follow-ups — can be fully automated using a well-designed LangGraph architecture and carefully engineered prompts. The key innovation is replacing all hard-coded boolean flow-control flags with prompt-driven LLM reasoning over conversation history, making the system genuinely intelligent and extensible.

8.2 ROI Analysis

Benefit	Estimated Impact
Triage time eliminated	Every inbound message is classified and routed in < 3 seconds vs. 2–5 minutes for a human agent reading and triaging.
Ticket management automated	Ticket creation, duplicate checking (Scenario B), and status lookup are fully automated — no agent intervention required for ~80% of interactions.
24/7 availability	The system handles customer queries at any hour without staffing costs. Night-time and weekend interactions no longer queue until the next business day.
Consistent response quality	LLM-generated responses maintain consistent tone, empathy, and accuracy regardless of agent mood, fatigue, or experience level.
Built-in quality monitoring	LLM-as-judge evaluation on every interaction provides continuous quality oversight at zero additional human cost.

8.3 Future Roadmap

Near-Term (Next 3 Months)

- Integrate with a production database (PostgreSQL) replacing SQLite for concurrent multi-user support.
- Add ticket status update capability — allow agents to mark tickets In Progress or Resolved via the admin dashboard.
- Implement email/SMS notification when a ticket is created or updated, using SendGrid or Twilio.
- Add multi-language support using GPT-4o's native multilingual capabilities.

Medium-Term (3–6 Months)

- Connect to a real banking CRM (e.g., Salesforce Service Cloud) via API for live ticket synchronization.
- Implement a RAG (Retrieval-Augmented Generation) layer — embed the bank's knowledge base (FAQs, product guides) so the Query Handler answers general questions with factual, bank-specific information.
- Build an automated prompt improvement loop: use thumbs-down interactions to fine-tune prompts or create few-shot examples for low-scoring categories.
- Add a Voice interface layer (Twilio + Whisper) to handle phone support interactions through the same agent pipeline.

Long-Term (6–12 Months)

- Introduce a Supervisor Agent pattern — a top-level LLM that monitors agent quality in real-time and can intervene or hand off to a human agent when confidence is low.
- Deploy on a production cloud infrastructure (AWS / Azure) with proper secrets management, load balancing, and session persistence.
- Implement agent fine-tuning using the interaction_logs dataset — use high-quality (5/5 rated) interactions as training examples to improve classification and response generation.
- Expand to additional banking channels: in-app chat widget, WhatsApp Business API, and web chat embed.

8.4 Technical Stack Summary

Component	Technology
Agent Orchestration	LangGraph (StateGraph, MemorySaver, conditional edges)
LLM	GPT-4o via LangChain ChatOpenAI (temperature=0)
Database	SQLite 3 (customer_details, support_tickets, interaction_logs)
UI Framework	Streamlit (chat interface, evaluation runner, logs dashboard)
Language	Python 3.11+
Key Libraries	langgraph, langchain, langchain-openai, langchain-core, streamlit, python-dotenv
Memory	LangGraph MemorySaver (in-process, per-session thread_id)
Evaluation	LLM-as-judge (second GPT-4o call) + manual test scenario runner

— End of Report —